

i Forside

UNIVERSITETET I OSLO

Det matematisk-naturvitenskapelige fakultet

Written exam in IN1900 2025 Fall

Date and time: Wednesday 17/12 from 09:00 to 13.00 (4 hours)

Permitted aides: None

A calculator is available in Inspira

It is important that you read this information carefully before you start answering the questions.

If you need to zoom in the exam set, hold ctrl and push + eller - on the numeric keyboard.

The exam contains multiple choice questions and text questions where you shall write short programs or read programs and write the output from the program. There are 16 questions to be answered, and in total 75 points available on these questions. Question 17 shall not be answered, but is used by the examiners during the grading of the exam, to include the points from the mid term exams.

The number of points available is specified for each question. For questions with multiple sub-questions, each sub-question has the same number of points. On multiple choice questions you get the same score (0) for a wrong answer and for not answering at all, so you should always mark an answer.

If you are missing information you can make your own reasonable assumptions, as long as they are in line with the "nature" of the question. In text questions you should then specify the assumptions you made, for instance in comments to the code.

All code in the question texts is written in Python 3.

Most of the questions lead to short code with little need for comments, unless you do something complicated or non-standard (which is not recommended; but in this case the comments should explain the ideas behind the program to make it easier to evaluate the code).

A question may ask you to write a function. A main program which calls the function is in this case not needed, unless it is specifically asked for in the question text.

1 Hva skrives ut?

What is printed in the terminal window when this code is run?

```
first_names = ['Minch', 'Han', 'Luke']
```

```
last_names = ['Yoda', 'Solo', 'Skywalker']
```

```
names = [last + first for first, last in zip(first_names, last_names)]
```

```
print(names[-1])
```

Select one alternative:

- ☐ En feilmelding
- ☐ LukeSkywalker
- ☐ SkywalkerLuke
- ☐ SoloSkywalker

Maximum marks: 2

2 Hva skrives ut?

What is printed in the terminal window when this code is run?

```
for i in range(3):  
    print(i, end=' '  
    for j in range(i):  
        print(j, end = ' ')
```

The argument `end = ' '` to the print function makes each output end with a space (' ') instead of a linebreak.

Select one alternative:

- ☐ 0 0 1 0 1 2 0 1 2
- ☐ 0 1 0 1 2
- ☐ 1 2 1 2 3
- ☐ 0 1 0 2 0 1

Maximum marks: 2

3 Løkke som teller

What is the value of the variable `n` when these program statements have been executed?

```
n = 0  
k = 10  
while n < k:  
    n = n + 2  
    k = k + 1
```

Select one alternative:

- ☐ 10
- ☐ 12
- ☐ 14
- ☐ 16
- ☐ 18
- ☐ 20
- ☐ 22
- ☐ 24

Maximum marks: 2

4 Hvilken linje mangler?

The function `diff2nd(f,x,h)` defined below computes an estimate of the second derivative of a function `f` in a point `x`.

```
def diff2nd(f, x, h=1E-6):
    r = (f(x-h) - 2*f(x) + f(x+h))/(h*h)
    return r
```

In the code defined below we want to use the function to estimate the second derivative of

$$f(x) = 2x^3 - 3x$$

for `x = 1.5`, and to study how the estimate changes as we reduce `h`.

```
for k in range(1,8):
    h = 10**(-k)
    #Missing line goes here
    print(f'h={h}: {d2g}')
```

One line is missing in the code. Which of the following lines should be inserted to make the code do what we want?

Select one alternative:

- ☐ `f = diff2nd(f=2*x**3-3*x, 1.5, h)`
- ☐ `d2g = diff2nd(f=2*x**3-3*x, 1.5)`
- ☐ `d2g = diff2nd(lambda x: 2*x**3-3*x, 1.5, h)`
- ☐ `d2g = diff2nd(2*x**3-3*x, 1.5, h)`

Maximum marks: 2

5 Hva skrives ut?

What is printed in the terminal when this program is run?

```
class F:
    def __init__(self, alpha, beta):
        self.alpha = alpha
        self.beta = beta

    def __call__(self, x):
        return self.alpha + self.beta * x

    def __add__(self, obj):
        return F(self.alpha+obj.alpha, self.beta+obj.beta)

a = F(0,1)
b = F(2,3)
c = (a + b)(2)
print(c)
```

Select one alternative:

- ☐ c
- ☐ 10
- ☐ 12
- ☐ 18

Maximum marks: 2

6 Hva skrives ut?

What is printed in the terminal window when this code is run?

```
def squared_sum(n):  
    s = 0  
    for i in range(n):  
        s += i**2  
    return s  
  
def test_squared_sum():  
    expected = 1 + 4 + 9 + 16  
    computed = squared_sum(4)  
    msg = f'expected {expected}, got {computed}'  
    assert expected == computed, msg  
  
test_squared_sum()
```

Select one alternative:

- ☐ Success: expected 30, got 30
- ☐ Nothing is printed
- ☐ NameError: name 'n' is not defined
- ☐ AssertionError: expected 30, got 14

Maximum marks: 2

7 Hva skrives ut?

What is printed in the terminal window when this code is run?

```
def find(text, args):  
    result = {}  
    for a in args:  
        result[a] = 0  
        for j in range(len(text)):  
            if text[j:j+len(a)] == a:  
                result[a] += 1  
    return result
```

```
text = "all work and no play makes jack a dull boy"
```

```
args = ['a', 'work', 'dull']
```

```
print(find(text, args))
```

Fill in your answer here

Format

B

I

U

$\times_2$

$\times^2$

I_x

Words: 0

Maximum marks: 2

8 Volum av et prisme

The volume of a rectangular prism is given by the formula $V = l \cdot b \cdot h$, where l , b og h are, respectively, the length, width and height of the prism.

The function `test_volume()` tests a function for calculating the volume of a prism:

```
def test_volume():  
    l, b, h = 2, 3, 4  
    lbh = [l, b, h]  
    expected = 24  
    computed = volume(lbh)  
    assert expected == computed
```

Write the function `volume` so that the test passes.

Fill in your answer here

1	
---	--

Maximum marks: 3

9 Stykkvis lineær funksjon

a) Write a Python function `piecewise(x, a, b)`, which implements the following mathematical function:

$$f(x) = \begin{cases} 0 & \text{for } x < a \\ \frac{x-a}{b-a} & \text{for } a \leq x < b \\ 1 & \text{for } x \geq b \end{cases}$$

Make the function as simple as possible. Avoid unnecessary if-tests.

b) Write a test function that tests the function from a). Choose $a = 0$ and $b = 1$, and test for three values of x ; one in each interval.

Fill in your answer here

1	
---	--

Maximum marks: 8

10 Implementasjon av en sum

The hyperbolic sine function **$\sinh x$** has the following series expansion:

$$\sinh x \approx \sum_{n=0}^N \frac{x^{2n+1}}{(2n+1)!}$$

Write a Python function **$\sinh(x,N)$** that for a given x and N calculates and returns the sum given above. The factorial function (!) is found in the math module and has the name **factorial**.

Fill in your answer here

1	
---	--

Maximum marks: 4

11 Filhåndtering og dictionaries

We want to write a program that reads information about cities, from files on the following format:

```
Oslo;Norway;698660
Stockholm;Sweden;975904
Helsinki;Finland;656254
Bergen;Norway;294029
Gothenburg;Sweden;608993
```

The columns are separated by semicolon (";") and represent the name of the city, the country where the city is located, and the population number. All the lines are on the same format, and there are no blank lines. The number of lines in the file can vary.

a) Write a function **read_city_file(filename)** that takes a filename as argument, reads a file on the format above, and returns a nested dictionary. The keys in the dictionary shall be the country, and the value for each key shall be a new dictionary that contains all the cities in the given country, with the name of the city as key and the population number as value.

b) Write a function **write_city_file(country_dict, filename)**, where the arguments are a dictionary on the format from a) and a filename. The function shall write the information in the dictionary to the file, with the same format as the file described above. You do not need to think about the order the cities are written in the file.

c) Write a function **filter_cities(country_dict, min_pop)** that takes two arguments; a dictionary on the format from a) and a population number. The function shall return a dictionary on the same format as in a), but where all cities with population < min_pop have been removed. (The function shall return a new dictionary, without changing the argument country_dict.)

Fill in your answer here

1	
---	--

Maximum marks: 12

12 Vektorer i 3D

We want to implement a class **Vec3D** that represents vectors in space. The class shall support vector addition and multiplication with a number (float or integer). Use of the class is illustrated in the code below:

```
>>> u = Vec3D([1.0, 2.0, 3.0])
>>> v = Vec3D([0.5, 1.5, 2.5])
>>> w = u + v
>>> print(w)
(1.5, 3.5, 5.5)
>>> s = u * 2.0
>>> print(s)
(2.0, 4.0, 6.0)
```

Write the code for the class **Vec3D**.

Hint 1: The code needs four special methods in total, including the constructor.

Hint 2: The result of adding two vectors shall be an instance of the **Vec3D** class. The same is true for the multiplication of a vector with a scalar.

Fill in your answer here

1	
---	--

Maximum marks: 10

13 Differenslikninger

We have the following system of difference equations for $n \geq 0$:

$$x_n = x_{n-1} + ax_{n-1} - bx_{n-1}y_{n-1},$$

$$y_n = y_{n-1} + dbx_{n-1}y_{n-1} - cy_{n-1}.$$

The system is a variant of the famous Lotka-Volterra model, which describes the dynamics in a population of predators and prey, where x_n is the number of prey and y_n the number of predators at a given time t_n . The parameters a, b, c and d are positive constants.

Write a function **LotkaVolterra(N, a, b, c, d, x0, y0)** which computes the solution (x_n, y_n) of the difference equation above for $n = 1, 2, \dots, N$. The other arguments to the function are the parameters a, b, c, d and the initial values x_0 and y_0 . The function shall return the solution as two arrays or lists containing the solution values $x_0, x_1, x_2, \dots, x_N$ and $y_0, y_1, y_2, \dots, y_N$.

Fill in your answer here

1	
---	--

Maximum marks: 5

14 Baklengs Euler

An ordinary differential equation (ODE) can be written on the general form: $u'(t) = f(t, u)$. The backward Euler method for solving such an ODE is given by the formula:

$$u_{n+1} = u_n + \Delta t f(t_{n+1}, u_{n+1})$$

If f is a non-linear function, this is a non-linear equation that must be solved for the unknown u_{n+1} for each time step. The equation is usually solved numerically with Newton's method or a similar solver.

In the simple case where $f(t, u) = -\lambda u$, the equation simplifies to:

$$u_{n+1} = u_n - \Delta t \lambda u_{n+1},$$

which can be rearranged to give an explicit update formula for u_{n+1} :

$$u_{n+1} = \frac{u_n}{1 + \Delta t \lambda}$$

Write a function **backward_euler_decay(lmbda, T, N, u0)** that applies this formula to solve $u'(t) = -\lambda u$ from $t=0$ to $t=T$, with N timesteps and initial value u_0 .

The function shall return two arrays or lists; one that contains the time points $t_0, t_1, \dots, t_N$, and one with the solution $u_0, u_1, \dots, u_N$.

Fill in your answer here

1	
---	--

Maximum marks: 5

15 Modelling med ODE-systemer

In this question we will solve a model for an outbreak of Ebola in Sierra Leone in 2014.

The population is divided into four groups; those that can be infected (S), those that are infected but have not yet developed the diseases, and can not yet infect others (E), those that are sick and can infect others (I), and those that have died from the disease (D). Let $S(t)$, $E(t)$, $I(t)$ and $D(t)$ be the number of people in each category. The following system of differential equations describes the dynamics of $S(t)$, $E(t)$, $I(t)$ and $D(t)$ in a time interval $[0, T]$:

$$S'(t) = -pS(t)I(t),$$

$$E'(t) = pS(t)I(t) - qE(t),$$

$$I'(t) = qE(t) - rI(t),$$

$$D'(t) = rI(t).$$

The constants p , q and r are assumed known, and all are > 0 .

a) Write a Python function **SEID(t, u)**, which defines the right hand side of the ODE system above. The function shall be possible to use with the ODESolver class hierarchy (attached).

The constants of the model can be local variables in the function, with values $p=0.0233$, $q = 1/10$ and $r = 1/10.38$. The arguments to the function are the time t (scalar) and an array or list u that contains the four components S, E, I and D at a given time point. The function shall return an array or list with containing the four derivatives.

b) Write a Python function **solve_SEID(T, N, S0, E0)**, which uses the function from a) and one of the solvers in the ODESolver hierarchy to solve the system from $t=0$ to $t=T$, with N time steps and initial conditions $S(0) = S0$, $E(0) = E0$, and $I(0) = D(0) = 0$. The function shall return 5 arrays:

- t , which holds the time points t_k where the numerical solution is calculated,
- S , which contains $S(0), S(t_1), \dots, S(t_n)$,
- E , which contains $E(0), E(t_1), \dots, E(t_n)$,
- I , which contains $I(0), I(t_1), \dots, I(t_n)$,
- D , which contains $D(0), D(t_1), \dots, D(t_n)$.

c) At the outbreak of the disease ($t=0$) the population in Sierra Leone was 5.48 million, and we want to solve the model for an extreme case where 10% of the population is infected and about to develop the disease. This gives $S0=4.93$ and $E0=0.55$. Write the code for calling the **solve_SEID** function with these initial conditions, $T=100$ and $N = 1000$. Add code to plot $I(t)$ and $D(t)$ in the same window, with a legend for each curve.

Maximum marks: 9

16 Baklengs Euler og ODESolver

The backward Euler method from question 14 can be used to solve general ODE systems where $f(t,u)$ is a non-linear function. The drawback of this method is that we do not have a simple formula for calculating u_{n+1} . Instead we must solve equations on the form

$$u_{n+1} = u_n + \Delta t f(t_{n+1}, u_{n+1})$$

for each time step. This is a system of non-linear equations that must be solved with Newton's method or a similar method. There are tools available for this task in Python, for instance the function **root** in the SciPy library, which can be imported with

```
from scipy.optimize import root
```

An example of using the function is

```
sol = root(f, x0)
```

Here, the argument **f** is a function ("callable") that takes an array as argument and returns an array of the same length. The function **f** shall take a single argument. **x0** is an array of start values (initial guess) for the solution algorithm, while the return value is an instance of the class `OptimizationResult`, which contains the solution of the system as an attribute **sol.x**. The attached file contains parts of the documentation for the function.

Use the **root** function to implement the backward Euler method as a subclass of the `ODESolver` class from the previous question.

Hint: You only need to implement a single method in the subclass, and it is possible to make it quite compact. However, if you find it easier to introduce one or more auxiliary methods to split up the implementation this is perfectly fine.

Fill in your answer here

1	
---	--

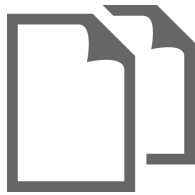
Maximum marks: 5

Write your answer in the box below. Changes are saved automatically.

Maximum marks: 25

Question 15

Attached



```

import numpy as np

class ODESolver:
    def __init__(self, f):
        self.f = lambda t, u: np.asarray(f(t, u), float)

    def set_initial_condition(self, u0):
        if np.isscalar(u0):
            # scalar ODE
            self.neq = 1
            # no of equations
            u0 = float(u0)
        else:
            # system of ODEs
            u0 = np.asarray(u0)
            self.neq = u0.size
            # no of equations
        self.u0 = u0

    def solve(self, t_span, N):
        # t_span: start and final time as a tuple/list of length 2
        # N: number of time steps
        t0, T = t_span
        self.dt = (T - t0) / N
        self.t = np.zeros(N + 1) # N steps ~ N+1 time points
        if self.neq == 1:
            self.u = np.zeros(N + 1)
        else:
            self.u = np.zeros((N + 1, self.neq))

        msg = "Please set initial condition before calling solve"
        assert hasattr(self, "u0"), msg

        self.t[0] = t0
        self.u[0] = self.u0

        for n in range(N):
            self.n = n
            self.t[n + 1] = self.t[n] + self.dt
            self.u[n + 1] = self.advance()
        return self.t, self.u

class ForwardEuler(ODESolver):
    def advance(self):
        u, f, n, t = self.u, self.f, self.n, self.t
        dt = self.dt
        return u[n] + dt * f(t[n], u[n])

class RungeKutta4(ODESolver):
    def advance(self):
        u, f, n, t = self.u, self.f, self.n, self.t
        dt = self.dt
        dt2 = dt / 2.0
        k1 = f(t[n], u[n],)
        k2 = f(t[n] + dt2, u[n] + dt2 * k1, )
        k3 = f(t[n] + dt2, u[n] + dt2 * k2, )
        k4 = f(t[n] + dt, u[n] + dt * k3, )
        return u[n] + (dt / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)

```

Question 16

Attached



`scipy.optimize.`

root

`root(fun, x0)`

Find a root of a vector function.

Parameters:

fun : *callable*

A vector function to find a root of.

Suppose the callable has signature `f0(x, *my_args, **my_kwargs)`, where `my_args` and `my_kwargs` are required positional and keyword arguments.

Rather than passing `f0` as the callable, wrap it to accept only `x`; e.g., pass `fun=lambda x: f0(x, *my_args, **my_kwargs)` as the callable, where `my_args` (tuple) and `my_kwargs` (dict) have been gathered before invoking this function.

x0: *ndarray*

Initial guess.

Returns:

sol : *OptimizeResult*

The solution represented as a `OptimizeResult` object. Important attributes are: `x` the solution array, `success` a Boolean flag indicating if the algorithm exited successfully and `message` which describes the cause of the termination. See `OptimizeResult` for a description of other attributes.

See also

`show\_options`

Additional options accepted by the solvers